

GitHub 연결하기

내 코드를 클라우드에 저장해줘

AI 처음으로 일 시키기 · FlowCoder

왜 GitHub이 필요해요?

시켜보기 ⑤에서 포트폴리오 웹페이지를 만들었죠.
근데 이건 **내 컴퓨터 안에만** 있습니다.

- 💥 컴퓨터 고장 → 다 날아감
- 🌐 다른 사람한테 보여주기? → 불가능
- 🚀 Vercel로 배포하려면? → **GitHub이 필수**

GitHub은 **부록 B·C로 가는 관문**입니다.

GitHub = "코드 버전의 Google Drive"

Google Drive	GitHub
문서·사진 저장	코드 저장
자동 버전 기록	커밋 이력 (변경 추적)
공유 링크	레포지토리 URL
협업 (동시 편집)	풀 리퀘스트

전 세계 개발자 1억 명이 쓰는 표준. 무료 개인 계정으로 시작.

오늘의 플로우



가입
github.com
5분



로그인 연결
gh auth login
30초



Claude Code가
나머지 전부
repo · push
자동

당신의 타이핑: 한 문장. 나머진 AI.

1단계: GitHub 가입

- 1. [github.com](#) 접속 → [Sign up](#)
- 2. 이메일 입력 → 비밀번호 → [유저네임](#) 정하기
 - 영문·숫자·하이픈만. 예: `soomin-kim`
 - 나중에 URL이 됩니다: `github.com/내유저네임`
- 3. 이메일 인증 코드 입력 → 완료

유저네임은 **신중히** — 프로필 URL로 계속 쓰이고, 변경이 번거롭습니다.

2단계: 2FA 설정 (강력 권장)

GitHub은 2026년 기준 **2FA를 거의 필수**로 요구합니다.

- 우측 상단 프로필 → **Settings** → **Password and authentication**
- Enable two-factor authentication** 클릭
- 방식 선택: **Authenticator app** 추천 (Google Authenticator, Authy 등)
- QR 코드 스캔 → 6자리 입력 → 복구 코드 **꼭 저장**

⚠ 복구 코드를 잃으면 계정 복구 매우 어렵습니다. 메모앱·비밀번호 매니저에 보관.

3단계: GitHub CLI 설치

Claude Code한테 시키면 끝:

```
GitHub CLI(gh)를 설치해줘.  
내 OS에 맞는 방법으로.
```

Claude Code가 알아서:

- 📁 Windows → `winget install GitHub.cli`
- 🍏 macOS → `brew install gh`

설치 확인: `gh --version`

4단계: GitHub 계정 연결

터미널에서:

```
gh auth login
```

질문이 몇 개 뜨는데, 전부 **엔터**로 기본값 선택:

- GitHub.com (기본)
- HTTPS (기본)
- Login with a web browser (기본)

→ 브라우저에 **8자리 코드** 입력 화면이 뜹니다.
터미널에 표시된 코드 복사 → 붙여넣기 → **Authorize** 클릭.

5단계: 핵심 한 문장

Claude Code한테 맡깁니다:

```
claude-workspace
```

폴더를 GitHub에 올려줘.

조건:

- private 레포로 만들어
- .gitignore 먼저 만들어서 민감 파일 제외
- README.md도 폴더 구조 보기 좋게 정리
- 첫 커밋 메시지는 "Initial setup"

이 한 덩어리가 원래라면 10단계짜리 작업입니다.

Claude Code가 하는 일

- 1. `git init` — 버전 관리 시작
- 2. `.gitignore` 생성 — `.env`, `node_modules` 등 제외
- 3. `README.md` 자동 작성 — 폴더 구조·용도 설명
- 4. `git add .` + 첫 커밋
- 5. `gh repo create` — GitHub에 private 레포 생성
- 6. `git push` — 업로드
- 7. 레포 URL을 터미널에 표시

 약 1분

결과 확인

터미널에 뜬 URL 클릭 → 브라우저에 내 레포:

✅ <https://github.com/soomin-kim/claude-workspace>

claude-workspace (private)

```
├── CLAUDE.md
├── README.md
├── SECURITY.md
├── .gitignore
├── tasks/
│   ├── todo.md
│   └── progress.md
```

🔒 Private repository
Just now · 1 commit

내 코드가 이제 인터넷에 안전하게 저장됨.

Git 핵심 3개념

외울 필요 없습니다. Claude Code가 다 합니다.

하지만 **개념은 알아야** 제대로 시킬 수 있어요.

커밋 · 브랜치 · 머지



커밋 Commit

게임 세이브 포인트

지금 상태를 "찰각" 저장.
이름표도 붙임.
"로그인 버튼 추가"



브랜치 Branch

평행우주 실험실

본체(main) 안 건드리고
새 갈래에서 실험.
망해도 본체는 안전



머지 Merge

실험 → 본체 합치기

성공한 실험을
본체에 반영.
실패한 건 그냥 삭제

작업 사이클 한눈에



main 커밋 feature 커밋 | 머지 커밋

- **main** = 본체. 안전하게 지킬 것
- **feature/xxx** = 실험 브랜치. 자유롭게 커밋
- 완성되면 **머지** → main이 업데이트됨

이 모든 게 한 문장으로

원래는 Git 명령 10개 외워야 하는 작업:

새 브랜치에서 포트폴리오 배경을 다크모드로 바꿔보고, 잘 되면 main에 머지하고 GitHub에도 올려줘.

Claude Code가 순서대로:

- `feature/dark-mode` 브랜치 생성
- CSS 수정 → 로컬에서 확인
- 커밋: "Add dark mode styles"
- main으로 돌아가 머지
- `git push` → GitHub 반영

실패하면? "방금 거 되돌려줘" — Claude Code가 복원.

이제 가능해진 것들

- ✓ 자동 백업 — 컴퓨터 고장? 다른 PC에서 `git clone` 한 줄이면 복구
- ✓ 변경 이력 추적 — "어제 뭐 바꿨더라?" → 커밋 로그 확인
- ✓ Vercel 배포 연결 — 부록 B에서 1분 만에 인터넷 공개
- ✓ 팀 협업 — 다른 사람 초대 → 동시 작업

다음 명령도 한 문장:
"Claude Code야, 내가 수정한 파일 커밋하고 GitHub에 올려줘"

부록 A 완료

다음: [부록 B · Vercel](#)

내 포트폴리오를 인터넷에 띄워줘

Speaker notes